

Blocco 3 - Casi di studio

Problemi guidati per ragionare su normalizzazione e schema

Relational Databases & SQL

Materiale aggiuntivo

Uso in aula

Questi casi non sostituiscono l'esercitazione principale: servono per ragionare insieme sul procedimento.

Obiettivo del materiale

- ▶ partire da tabelle piatte realistiche;
- ▶ riconoscere granularità, duplicazioni e anomalie;
- ▶ formulare dipendenze funzionali in linguaggio naturale;
- ▶ separare fatti, entità e dati storici;
- ▶ confrontare soluzioni accettabili e soluzioni fragili.

Procedimento da ripetere

- ➊ leggere una riga come frase: “una riga rappresenta ...”;
- ➋ evidenziare informazioni ripetute;
- ➌ chiedere quali attributi determinano altri attributi;
- ➍ separare relazioni con identità propria;
- ➎ conservare chiavi e riferimenti;
- ➏ verificare che il report iniziale sia ricostruibile.

Domande guida per la classe

- ▶ Che cosa succede se aggiorno questo dato in una sola riga?
- ▶ Posso inserire questo fatto senza inventare altri fatti?
- ▶ Posso cancellare una riga senza perdere informazioni indipendenti?
- ▶ Questo attributo descrive la riga, oppure descrive un'altra entità?
- ▶ Questo valore deve fotografare il passato o riflettere lo stato corrente?

Caso 1: Segreteria Corsi

Una tabella iscrizioni mescola studenti, corsi, docenti ed edizioni annuali.

Caso 1: problema informale

Contesto

La segreteria didattica esporta un file con una riga per iscrizione a un corso. Il file è comodo per controllare chi è iscritto, ma contiene anche dati anagrafici, informazioni sul corso e dati del docente.

Obiettivo

Separare ciò che appartiene allo studente, al corso, all'edizione del corso e all'iscrizione.

Caso 1: tabella iniziale

matricola	studente	email	corso	titolo	docente
S01	Sara Bianchi	sara@mail	DB101	Database	Neri
S01	Sara Bianchi	sara@mail	AI201	AI base	Rossi
S02	Marco Verdi	marco@mail	DB101	Database	Neri
S03	Luca Neri	luca@mail	NET10	Reti	Conti

Granularità iniziale

Una riga rappresenta l'iscrizione di uno studente a una specifica offerta didattica.

Caso 1: anomalie da far emergere

- ▶ il nome e l'email dello studente sono ripetuti per ogni iscrizione;
- ▶ il titolo del corso è ripetuto per ogni studente iscritto;
- ▶ se cambia il docente di una edizione, bisogna aggiornare molte righe;
- ▶ non posso registrare una nuova edizione senza almeno uno studente iscritto;
- ▶ cancellando l'unica iscrizione a un corso potrei perdere informazioni sul corso.

Caso 1: procedimento

- ▶ `matricola` determina nome ed email dello studente;
- ▶ `codice_corso` determina il titolo generale del corso;
- ▶ il docente non descrive sempre il corso: può descrivere l'edizione;
- ▶ voto, data iscrizione e stato descrivono l'iscrizione;
- ▶ la chiave dell'iscrizione può essere (`student_id`, `edition_id`).

Caso 1: soluzione proposta

- ▶ `students(student_id, matricola, name, email)`
- ▶ `courses(course_id, code, title)`
- ▶ `teachers(teacher_id, name, email)`
- ▶ `course_editions(edition_id, course_id, academic_year, teacher_id)`
- ▶ `enrollments(student_id, edition_id, enrolled_at, status, grade)`

Punto didattico

“Corso” ed “edizione del corso” non sono la stessa cosa: distinguere i livelli evita dipendenze sbagliate.

Caso 2: Manutenzione Impianti

Una tabella interventi contiene impianti, sedi, tecnici e ricambi usati.

Caso 2: problema informale

Contesto

Un reparto manutenzione usa un foglio unico per registrare interventi tecnici sugli impianti. Quando in un intervento vengono usati più ricambi, la riga viene duplicata cambiando solo il ricambio.

Obiettivo

Capire quali dati descrivono l'intervento e quali descrivono impianto, tecnico, sede o ricambio.

Caso 2: tabella iniziale

intervento	data	impianto	sede	tecnico	ricambio
I100	12/05	COMP-01	Roma	Bianchi	Filtro A
I100	12/05	COMP-01	Roma	Bianchi	Guarnizione B
I101	13/05	PUMP-07	Napoli	Verdi	Olio X
I102	13/05	COMP-01	Roma	Neri	Filtro A

Granularità iniziale

La riga non rappresenta davvero un intervento: rappresenta un ricambio usato dentro un intervento.

Caso 2: anomalie da discutere

- ▶ i dati dell'intervento sono ripetuti per ogni ricambio;
- ▶ il nome della sede è ripetuto per ogni impianto e intervento;
- ▶ un intervento senza ricambi non ha un posto naturale nella tabella;
- ▶ cancellando l'ultima riga di un intervento si perde l'intervento;
- ▶ il costo totale può essere calcolato in modo incoerente se si sommano righe duplicate.

Caso 2: procedimento

- ▶ prima si corregge la granularità: una relazione per intervento e una per ricambio usato;
- ▶ l'impianto ha identità propria e appartiene a una sede;
- ▶ il tecnico ha identità propria e può partecipare a molti interventi;
- ▶ il ricambio ha descrizione e codice separati dalla quantità usata;
- ▶ il costo del ricambio usato può essere storico e stare nella riga di utilizzo.

Caso 2: soluzione proposta

- ▶ `sites(site_id, name, city)`
- ▶ `assets(asset_id, asset_code, site_id, model)`
- ▶ `technicians(technician_id, name, team)`
- ▶ `maintenance_jobs(job_id, asset_id, technician_id, opened_at, closed_at, job_type)`
- ▶ `spare_parts(part_id, part_code, description)`
- ▶ `job_parts(job_id, part_id, quantity, unit_cost)`

Caso 3: Fatture Fornitore

Una tabella fatture forza a distinguere dati correnti e dati storici.

Caso 3: problema informale

Contesto

L'amministrazione conserva fatture di acquisto in una tabella unica. La stessa riga contiene fornitore, indirizzo, articolo, categoria, condizioni di pagamento e importo.

Obiettivo

Normalizzare senza cancellare informazioni storiche necessarie per ricostruire la fattura originale.

Caso 3: tabella iniziale

fattura	fornitore	p. IVA	articolo	quantità	prezzo
F-77	Alfa Srl	IT001	Cavo 2m	10	4.50
F-77	Alfa Srl	IT001	Router B	2	120.00
F-78	Beta Spa	IT099	Sensore A	5	35.00
F-79	Alfa Srl	IT001	Cavo 2m	20	4.30

Domanda critica

Il prezzo e l'indirizzo del fornitore devono riflettere lo stato corrente o la fattura emessa in passato?

Caso 3: anomalie e rischio opposto

- ▶ duplicazione di fornitore e articolo;
- ▶ aggiornamento fragile dell'anagrafica fornitore;
- ▶ cancellazione di una riga può far perdere una fattura;
- ▶ normalizzare troppo può perdere il prezzo effettivo fatturato;
- ▶ usare solo dati correnti può rendere impossibile ricostruire il documento originale.

Caso 3: procedimento

- ▶ separare fornitore, fattura, righe fattura e prodotti;
- ▶ tenere nella riga fattura quantità, prezzo e descrizione al momento della fattura;
- ▶ decidere se indirizzo e ragione sociale devono essere snapshot storici;
- ▶ distinguere listino corrente da prezzo fatturato;
- ▶ documentare le denormalizzazioni motivate dalla storia del documento.

Caso 3: soluzione proposta

- ▶ `suppliers(supplier_id, vat_number, current_name, current_address)`
- ▶ `products(product_id, sku, current_name, category)`
- ▶ `purchase_invoices(invoice_id, supplier_id, invoice_number, invoice_date, supplier_name_snapshot, address_snapshot)`
- ▶ `purchase_invoice_lines(invoice_id, line_no, product_id, description_snapshot, quantity, unit_price)`

Punto didattico

La normalizzazione riduce dipendenze sbagliate; non obbliga a cancellare gli snapshot storici quando sono parte del fatto.

Caso 4: Biblioteca

Prestiti, copie fisiche, libri e autori non stanno bene in una sola tabella.

Caso 4: problema informale

Contesto

Una biblioteca registra prestiti in un file unico. Alcuni libri hanno più autori e lo stesso libro può avere più copie fisiche con barcode diversi.

Obiettivo

Distinguere opera bibliografica, copia fisica, autore, lettore e prestito.

Caso 4: tabella iniziale

prestito	lettore	ISBN	titolo	autore	copia
P01	Sara	978-1	SQL Base	Neri	C001
P02	Marco	978-1	SQL Base	Neri	C002
P03	Sara	978-2	Data Model	Rossi; Conti	C010
P04	Luca	978-3	Python	Verdi	C011

Problema nascosto

La colonna autore contiene talvolta più valori: non è un attributo atomico.

Caso 4: anomalie da discutere

- ▶ il titolo del libro è ripetuto per ogni copia e prestito;
- ▶ gli autori multipli non sono rappresentati come righe distinte;
- ▶ una copia fisica ha stato e collocazione propri;
- ▶ non posso registrare una nuova copia non ancora prestata;
- ▶ cancellando l'ultimo prestito potrei perdere l'esistenza del libro.

Caso 4: procedimento

- ▶ ISBN identifica il libro, non la copia fisica;
- ▶ barcode identifica la copia, che può essere prestata molte volte;
- ▶ autore e libro sono in relazione multi-a-molti;
- ▶ il prestito collega lettore e copia in un intervallo temporale;
- ▶ data prestito e data restituzione descrivono il prestito.

Caso 4: soluzione proposta

- ▶ `readers(reader_id, fiscal_code, name, email)`
- ▶ `books(book_id, isbn, title, publisher)`
- ▶ `authors(author_id, name)`
- ▶ `book_authors(book_id, author_id, author_order)`
- ▶ `book_copies(copy_id, book_id, barcode, shelf, status)`
- ▶ `loans(loan_id, copy_id, reader_id, borrowed_at, returned_at)`

Caso 5: Energia

Lecture, POD, contratti, contatori e tariffe hanno granularità diverse.

Caso 5: problema informale

Contesto

Un operatore energia riceve un file mensile con consumi elettrici. Ogni riga contiene cliente, punto di fornitura, contatore, tariffa e lettura mensile.

Obiettivo

Distinguere cliente, punto di fornitura, contratto, contatore installato e lettura energetica.

Caso 5: tabella iniziale

mese	cliente	POD	contatore	tariffa	kWh
2026-01	Enel Plant A	IT001E	M-77	BTA6	18420
2026-02	Enel Plant A	IT001E	M-77	BTA6	17610
2026-01	Store Roma	IT009E	M-90	BTA3	2480
2026-01	Store Roma	IT010E	M-91	BTA3	1930

Granularità iniziale

Una riga rappresenta una lettura mensile di un contatore associato a un punto di fornitura.

Caso 5: anomalie da discutere

- ▶ i dati cliente e tariffa sono ripetuti a ogni mese;
- ▶ il POD identifica un punto di fornitura, non il cliente;
- ▶ un cliente può avere più POD;
- ▶ un contatore può essere sostituito nel tempo sullo stesso POD;
- ▶ cambiare tariffa non deve modificare le letture storiche già fatturate.

Caso 5: procedimento

- ▶ separare soggetti stabili: cliente, POD, contatore e piano tariffario;
- ▶ rappresentare il contratto come relazione temporale tra cliente e POD;
- ▶ rappresentare l'installazione del contatore come relazione temporale tra POD e contatore;
- ▶ mettere mese, energia misurata e qualità della misura nella lettura;
- ▶ tenere eventuale tariffa applicata come riferimento storico del contratto.

Caso 5: soluzione proposta

- ▶ `customers`: cliente, ragione sociale, segmento
- ▶ `supply_points`: POD, indirizzo, tensione, stato
- ▶ `meters`: matricola contatore, modello, classe
- ▶ `meter_installations`: POD, contatore, inizio, fine
- ▶ `tariff_plans`: codice tariffa, descrizione, validità
- ▶ `supply_contracts`: cliente, POD, tariffa, inizio, fine
- ▶ `energy_readings`: installazione, mese, kWh, qualità misura

Caso 6: Monitoraggio Ambientale

Misure, stazioni, sensori e parametri non hanno la stessa identità.

Caso 6: problema informale

Contesto

Un ente ambientale raccoglie misure orarie da stazioni distribuite sul territorio. Il file unico ripete posizione, sensore, parametro misurato, unità e soglia normativa.

Obiettivo

Separare stazione, sensore, parametro ambientale, installazione e misura osservata.

Caso 6: tabella iniziale

timestamp	stazione	comune	sensore	parametro	valore
10:00	RM-Centro	Roma	S-101	PM10	42
11:00	RM-Centro	Roma	S-101	PM10	45
10:00	RM-Centro	Roma	S-202	NO2	31
10:00	VT-Nord	Viterbo	S-301	PM10	28

Granularità iniziale

Una riga rappresenta una misura di un parametro in un istante, prodotta da un sensore installato in una stazione.

Caso 6: anomalie da discutere

- ▶ posizione e comune della stazione sono ripetuti per ogni misura;
- ▶ unità e soglia del parametro sono ripetute per ogni osservazione;
- ▶ un sensore può essere sostituito o spostato;
- ▶ non posso registrare una stazione senza misure;
- ▶ una calibrazione del sensore non dovrebbe modificare le misure storiche.

Caso 6: procedimento

- ▶ identificare la stazione come luogo di osservazione;
- ▶ identificare il sensore come dispositivo fisico con matricola;
- ▶ identificare il parametro come tipo di grandezza misurata;
- ▶ modellare installazione e calibrazione come fatti temporali;
- ▶ lasciare timestamp, valore e stato qualità nella misura.

Caso 6: soluzione proposta

- ▶ `monitoring_stations`: codice, comune, coordinate, quota
- ▶ `sensors`: matricola, modello, produttore
- ▶ `environmental_parameters`: codice, nome, unità, soglia
- ▶ `sensor_installations`: stazione, sensore, parametro, inizio, fine
- ▶ `sensor_calibrations`: sensore, data, esito, note
- ▶ `measurements`: installazione, timestamp, valore, qualità

Confronto finale

I sei casi mostrano lo stesso metodo applicato a domini diversi.

Cosa confrontare tra le soluzioni

- ▶ qual è la granularità della relazione principale;
- ▶ quali attributi sono spostati perché dipendono da un'altra chiave;
- ▶ quali tabelle ponte servono per multi-a-molti;
- ▶ quali valori storici restano nella relazione evento;
- ▶ quali report iniziali devono essere ricostruibili con join.

Checklist per la discussione

- ❶ Ogni tabella ha una frase “una riga per ...”?
- ❷ Ogni chiave determina tutti gli attributi non chiave?
- ❸ Ci sono colonne con liste di valori?
- ❹ Ci sono attributi copiati solo per comodità?
- ❺ La soluzione mantiene la storia necessaria?
- ❻ Le anomalie iniziali sono state eliminate o almeno motivate?

- ▶ La normalizzazione parte dal significato delle righe, non dai nomi delle colonne.
- ▶ Una scomposizione è corretta se elimina dipendenze sbagliate e preserva i fatti necessari.
- ▶ Alcuni dati storici restano duplicati intenzionalmente: va dichiarato perché.